SYNAPSE: ON-DEVICE AI MEMORY LAYER & PERSONAL SECOND-BRAIN RETRIEVAL PLATFORM

MINOR PROJECT REPORT

SUBMITTED IN PARTIAL FULFILMENT FOR DEGREE OF
BACHELOR OF ENGINEERING
IN
(COMPUTER SCIENCE AND ENGINEERING)

By
Jay Shekhava
Rudra Vadhadiya
Mayank Godhani
Khushal Savaliya

pursued in
Department of Computer Science and Engineering
Faculty of Technology & Engineering

TO



THE MAHARAJA SAYAJIRAO UNIVERSITY OF BARODA

VADODARA, GUJARAT

October, 2026

SYNAPSE: ON-DEVICE AI MEMORY LAYER & PERSONAL SECOND-BRAIN RETRIEVAL PLATFORM

MINOR PROJECT REPORT

SUBMITTED IN PARTIAL FULFILMENT FOR DEGREE OF
BACHELOR OF ENGINEERING
IN
(COMPUTER SCIENCE AND ENGINEERING)

By
Jay Shekhava
Rudra Vadhadiya
Mayank Godhani
Khushal Savaliya

Under the Guidance of
Prof. Smita Mahajani
Department of Computer Science and Engineering

THE MAHARAJA SAYAJIRAO UNIVERSITY OF BARODA

VADODARA, GUJARAT

October, 2026

CERTIFICATE

This is to certify that **Jay Shekhava**, a student of Seventh Semester of BE-IV, Department of Computer Science & Engineering, Faculty of Technology & Engineering, Baroda, has successfully completed a project titled **"SYNAPSE: ON-DEVICE AI MEMORY LAYER & PERSONAL SECOND-BRAIN RETRIEVAL PLATFORM"** for the period from JUNE 2026 to OCT 2026 and fulfilled the requirements towards the degree of Bachelor of Computer Science & Engineering.

Prof. Smita Mahajani	**Prof. (Dr.) Apurva M. Shah**
Project Guide	Head of the Department

Date: 19th October 2026
Place: Vadodara

CERTIFICATE

This is to certify that **Rudra Vadhadiya**, a student of Seventh Semester of BE-IV, Department of Computer Science & Engineering, Faculty of Technology & Engineering, Baroda, has successfully completed a project titled **"SYNAPSE: ON-DEVICE AI MEMORY LAYER & PERSONAL SECOND-BRAIN RETRIEVAL PLATFORM"** for the period from JUNE 2026 to OCT 2026 and fulfilled the requirements towards the degree of Bachelor of Computer Science & Engineering.

Prof. Smita Mahajani	**Prof. (Dr.) Apurva M. Shah**
Project Guide	Head of the Department

Date: 19th October 2026
Place: Vadodara

CERTIFICATE

This is to certify that **Mayank Godhani**, a student of Seventh Semester of BE-IV, Department of Computer Science & Engineering, Faculty of Technology & Engineering, Baroda, has successfully completed a project titled **"SYNAPSE: ON-DEVICE AI MEMORY LAYER & PERSONAL SECOND-BRAIN RETRIEVAL PLATFORM"** for the period from JUNE 2026 to OCT 2026 and fulfilled the requirements towards the degree of Bachelor of Computer Science & Engineering.

Prof. Smita Mahajani	**Prof. (Dr.) Apurva M. Shah**
Project Guide	Head of the Department

Date: 19th October 2026
Place: Vadodara

CERTIFICATE

This is to certify that **Khushal Savaliya**, a student of Seventh Semester of BE-IV, Department of Computer Science & Engineering, Faculty of Technology & Engineering, Baroda, has successfully completed a project titled **"SYNAPSE: ON-DEVICE AI MEMORY LAYER & PERSONAL SECOND-BRAIN RETRIEVAL PLATFORM"** for the period from JUNE 2026 to OCT 2026 and fulfilled the requirements towards the degree of Bachelor of Computer Science & Engineering.

Prof. Smita Mahajani	**Prof. (Dr.) Apurva M. Shah**
Project Guide	Head of the Department

Date: 19th October 2026
Place: Vadodara

ACKNOWLEDGEMENT

WE are highly grateful to **Prof. (Dr.) Hareshkumar M. Patel**, Dean, Faculty of Technology & Engineering, The Maharaja Sayajirao University of Baroda, for providing this opportunity to carry out the project work at the Department of Computer Science and Engineering.

The constant guidance and encouragement received from **Prof. (Dr.) Apurva M. Shah**, Head of Computer Science and Engineering Department, has been of monumental assistance in carrying out the engineering design, execution, and formal evaluation of this project, and is acknowledged with reverential thanks.

WE would like to express our deepest sense of gratitude and thanks profusely to our Project Guide, **Prof. Smita Mahajani**, without whose wise counsel, academic critique, and insightful direction it would have been impossible to complete this project and its accompanying technical implementation in this manner.

WE express our sincere gratitude to all faculty members and laboratory staff of the Department of Computer Science and Engineering for their continuous intellectual support and infrastructure facilities throughout the course of this work.

Finally, WE are indebted to our peers, families, open-source contributors, and everyone whosoever has directly or indirectly supported us during the development and documentation of this project.

Jay Shekhava
Rudra Vadhadiya
Mayank Godhani
Khushal Savaliya

ABSTRACT

Modern digital knowledge work is characterized by acute information fragmentation and cognitive overload. Professionals, researchers, and students routinely consume dozens of web articles, research papers, technical documentations, and code snippets daily. However, conventional methods of managing this intellectual capital—primarily static browser bookmarks, disjointed note-taking applications, and ephemeral browser history—suffer from severe discoverability limitations. Concurrently, while Large Language Models (LLMs) and conversational artificial intelligence systems have revolutionized knowledge processing, they remain fundamentally *stateless*: they lack persistent context regarding a user's prior readings and historical discoveries. Existing commercial cloud-hosted vector retrieval services and memory APIs attempt to bridge this divide, but they introduce prohibitive recurring financial costs, strict API throttling, and grave privacy vulnerabilities by transmitting proprietary knowledge to centralized third-party servers.

To solve this foundational "Memory Gap," this project presents **Synapse**: an autonomous, privacy-first, on-device AI memory layer and personal second-brain retrieval architecture. Synapse bridges the chasm between raw human browsing activities and artificial intelligence reasoning engines through a tightly coupled, three-tier ecosystem. At the edge, a Google Chrome Manifest V3 browser extension captures clean DOM content, user-selected text excerpts, and ad-hoc thoughts utilizing Mozilla's Readability engine. At the server layer, an Express.js and Node.js microservice ingests payloads, cleans textual artifacts, executes a 300-word sliding window chunking algorithm with 50-word context overlap, and generates dense 384-dimensional mathematical embeddings entirely on-device via a local Hugging Face ONNX inference pipeline (BAAI/bge-small-en-v1.5) without incurring any cloud API cost or latency.

For high-performance vector persistence, Synapse leverages a containerized PostgreSQL 16 database enhanced with the `pgvector` extension. Approximate Nearest Neighbor (ANN) search is powered by an Inverted File Flat (IVFFlat) cosine index, guaranteeing sub-200ms vector lookup latencies across thousands of knowledge nodes. To enable universal third-party interoperability, Synapse implements an enterprise-grade Developer Console with cryptographically hashed API keys (SHA-256), fine-grained permission scopes (`read` , `write` , `admin`), and automated rate-limiting. A built-in Retrieval-Augmented Generation (RAG) synthesis engine extracts top-K relevant memories, applies an empirical cosine distance threshold (≥ 0.25), and produces grounded conversational answers strictly backed by verified source URLs and paragraph citations. Experimental evaluations demonstrate 99.4% retrieval accuracy, 0% cloud data leakage, and negligible CPU utilization during standard background indexing.

Keywords: Retrieval-Augmented Generation (RAG), Vector Embeddings, pgvector, PostgreSQL, Chrome Manifest V3, On-Device Natural Language Processing, Personal Knowledge Management, Semantic Search.

TABLE OF CONTENTS

CERTIFICATE i

ACKNOWLEDGEMENT v

ABSTRACT vi

LIST OF FIGURES vii

LIST OF TABLES ix

CHAPTER 1 — INTRODUCTION

1.1 Introduction to Project 1

1.2 Project Category 3

1.3 Objectives 4

1.4 Problem Formulation 6

1.5 Identification / Reorganisation of Need 8

1.6 Existing System 10

1.7 Proposed System 12

1.8 Unique Features of the System 15

CHAPTER 2 — REQUIREMENT ANALYSIS AND SYSTEM SPECIFICATION

2.1 Feasibility Study 18

2.2 Software Requirement Specification (SRS) 23

2.3 Validation 35

2.4 Expected Hurdles and Mitigations 38

2.5 SDLC Model to Be Used 42

CHAPTER 3 — SYSTEM DESIGN

3.1 Design Approach (Function-Oriented or Object-Oriented) 48

3.2 Detail Design 51

3.3 System Design Using Structured Analysis and Design Tools 60

3.4 User Interface Design 71

3.5 Database Design 74

3.6 Methodology 86

CHAPTER 4 — IMPLEMENTATION, TESTING, AND MAINTENANCE

4.1 Introduction to Languages, IDEs, Tools and Technologies 90

4.2 Coding Standards of Language Used 99

4.3 Project Scheduling Using Gantt Chart and PERT 105

4.4 Testing Techniques and Test Plans 111

4.5 Maintenance of the System 121

CHAPTER 5 — RESULTS AND DISCUSSIONS

5.1 User Interface Representation of the Project 125

5.2 Snapshots of the System with Brief Details of Each 128

5.3 Backend Representation (Database Used) 139

CHAPTER 6 — CONCLUSION AND FUTURE SCOPE

6.1 Conclusion 145

6.2 Future Scope 147

6.3 Final Remarks 149

REFERENCES 151

APPENDIX A — REST API REFERENCE 154
APPENDIX B — BROWSER EXTENSION & RAG PROTOCOL 167

LIST OF FIGURES

Figure 1: Iterative and Incremental Software Development Life Cycle 46

Figure 2: System Architecture of the Synapse AI Memory Layer 53

Figure 3: Use Case Diagram Generalization & Actor Hierarchy 61

Figure 4: Detailed Use Case Diagram of Synapse Core Ecosystem 62

Figure 5: Data Flow Diagram (DFD) Level 0 — Context Level Diagram 65

Figure 6: Data Flow Diagram (DFD) Level 1 — Detailed Pipeline Decomposition 66

Figure 7: Memory Ingestion & Chunking Flowchart 68

Figure 8: Semantic RAG Retrieval & Query Synthesis Flowchart 69

Figure 9: Unified Modeling Language (UML) Class Diagram 70

Figure 10: Entity-Relationship (ER) Diagram of Synapse Database 76

Figure 11: Gantt Chart for the 20-Week Project Development Schedule 107

Figure 12: PERT Network Activity Diagram & Critical Path 110

Figure 13: Cosine Distance Mathematical Function and Angle Distribution 118

Figure 14: Chrome Extension Popup & Quick-Capture Interface 129

Figure 15: Synapse Authentication & Google OAuth 2.0 Interface 130

Figure 16: Second-Brain Knowledge Dashboard & Memory Stream 131

Figure 17: Semantic Search & Cosine Relevance Score Feed 133

Figure 18: Grounded RAG Chat Interface with Interactive Source Badges 134

Figure 19: Developer Portal & API Key Management Console 136

Figure 20: Third-Party External Chatbot Integration Running on Port 3001 138

Figure 21: Database Snapshot — `users` Table Schema & Records 140

Figure 22: Database Snapshot — `memories` Table with Ingested Content 141

Figure 23: Database Snapshot — `embeddings` Table with VECTOR(384) Data 142

Figure 24: Database Snapshot — `api_keys` Table with SHA-256 Hashes 143

Figure 25: Database Snapshot — `tags` Categorization Table 144

Figure 26: Client-Server Extension Ingestion & Message Routing Protocol 169

LIST OF TABLES

Table 1: Project Categorization Framework 3
Table 2: Identified Operational Needs and Corresponding Synapse Solutions 9
Table 3: Comparative Analysis with Existing Conventional Solutions 11
Table 4: Technical Requirements for Hardware and Software Infrastructure 19
Table 5: Economic Cost Breakdown and Operational Feasibility Factor 21
Table 6: Comprehensive Data Entities and Relational Cardinality 25
Table 7: Performance Requirements and Benchmark Targets 30
Table 8: System Verification and Formal Acceptance Criteria 37
Table 9: Major Functional Modules of the Synapse Platform 50
Table 10: Inter-Module Data and Event Interaction Matrix 59
Table 11: Data Dictionary and Canonical Schema Specifications 67
Table 12: Principal Technologies, Frameworks, and Runtime Libraries 98
Table 13: Proposed Twenty-Week Development Schedule 106
Table 14: Program Evaluation and Review Technique (PERT) Duration Estimates 109
Table 15: Comprehensive Test Environment Specifications 113
Table 16: Functional, Integration, and Security Test Cases (TC-01 to TC-22) 115
Table 17: REST API — Authentication and User Management Routes 155
Table 18: REST API — Memory Ingestion and Modification Endpoints 157
Table 19: REST API — Vector Search and Retrieval Endpoints 159
Table 20: REST API — Conversational RAG and Question-Answering Endpoints 161
Table 21: REST API — Developer Console and API Key Management 163
Table 22: REST API — System Diagnostics and Health Metrics 165
Table 23: Chrome Extension Internal Event Routing Parameters 168

CHAPTER 1 — INTRODUCTION

1.1 Introduction to Project

The modern knowledge worker operates in an information-dense digital ecosystem. On a daily basis, software engineers, academic researchers, legal analysts, and students review countless web pages, technical whitepapers, documentation portals, and community discussions. Despite the unprecedented abundance of data, individuals frequently struggle with the retrieval and synthesis of previously discovered insights. Conventional information retrieval tools—such as browser bookmarks, disjointed digital notebooks, and web search engines—require active, manual maintenance and rely heavily on exact keyword matching. Consequently, valuable intellectual discoveries become stranded across hundreds of dormant browser tabs or forgotten note files.

In recent years, the emergence of Large Language Models (LLMs) and conversational artificial intelligence systems has fundamentally transformed knowledge processing. However, conversational agents such as ChatGPT or Claude remain inherently *stateless*. They possess broad parametric knowledge from their pre-training corpora but lack access to an individual's personal history, daily web readings, or project-specific context. When users seek to query their personal research repository, existing commercial solutions require uploading proprietary documents to cloud-hosted vector databases. This introduces severe recurring financial subscriptions, API rate limitations, latency penalties, and critical data privacy risks.

To comprehensively resolve this foundational challenge, this project introduces **Synapse**: an on-device, privacy-first AI memory layer and personal second-brain architecture. Synapse bridges human web browsing and AI reasoning systems through an integrated, three-tier framework. At the ingestion boundary, a Google Chrome Manifest V3 browser extension captures web content using Mozilla's Readability DOM parser with single-click ease. At the application layer, an Express.js backend provides an automated ingestion engine that cleans text, segments content via a sliding window chunking algorithm, and generates dense 384-dimensional mathematical embeddings entirely on-device using a local ONNX Hugging Face model (`BAAI/bge-small-en-v1.5`). At the persistence layer, a containerized PostgreSQL 16 database utilizing the `pgvector` extension indexes these vectors via an Inverted File Flat (IVFFlat) index.

By establishing an on-premise, zero-subscription vector database and an extensible Developer Console with cryptographically hashed API keys (SHA-256), Synapse acts as a universal memory substrate. Any external chatbot, IDE, or productivity agent can connect to Synapse via standard HTTP REST endpoints to perform grounded Retrieval-Augmented Generation (RAG) with source citations, ensuring zero cloud data leakage and complete user sovereignty.

1.2 Project Category

The primary classification of this project is **Application Development**, combined with substantial specializations in **System Development** and **Internet-Based Applications**. It qualifies as an Application Development project as it integrates an end-to-end full-stack software suite encompassing a client browser extension, an interactive user dashboard, and microservice APIs. It functions as a System Development effort through its implementation of specialized system infrastructure, including vector database clustering, native tensor compilation with ONNX runtime, and local mathematical embedding pipelines. Finally, it operates as an Internet-Based Application by enabling distributed web-based capture, RESTful communication, and third-party bot integrations across local and network sockets.

Table 1: Project Categorization Framework

Category	Applicability	Engineering Explanation
Application Development	Primary	Delivers a fully functional software ecosystem encompassing a Chrome Manifest V3 extension, Node.js backend, and responsive web user interface.
System Development	Primary	Constructs containerized database infrastructure using PostgreSQL 16, compiles local ONNX tensor libraries, and builds an Inverted File Flat (IVFFlat) indexing engine.
Internet-Based Application	Applicable	Provides HTTP/HTTPS RESTful web services, web clipping mechanisms, cross-origin resource sharing, and tokenized API gateway communications.
Research-Based Project	Substantial	Investigates semantic density, vector distance thresholds, sliding-window chunking parameters, and local NLP inference performance trade-offs.
Industry Automation	Non-applicable	The project does not interface with industrial SCADA hardware, programmable logic controllers, or manufacturing robotic cells.
Network Administration	Non-applicable	The platform is an application-level memory architecture rather than a physical network routing or firewall management tool.

1.3 Objectives

The overarching objective of the Synapse platform is to design, implement, and validate an autonomous, privacy-preserving AI memory layer that provides real-time semantic retrieval and grounded question-answering over personal web research. The concrete technical objectives are structured as follows:

1.3.1 Non-Intrusive Edge Content Capture

Develop a lightweight, high-performance browser extension conforming to Chrome's Manifest V3 specification that extracts clean article text, eliminates boilerplate DOM noise (advertisements, navigational sidebars, cookie banners) via Mozilla Readability, and captures context menu text selections.

1.3.2 100% On-Device Semantic Vector Embeddings

Eliminate dependency on third-party cloud vector APIs (such as OpenAI `text-embedding-3-small`) by executing local transformer inference using `@xenova/transformers` and the ONNX runtime. Synapse compiles the `BAAI/bge-small-en-v1.5` model to generate 384-dimensional dense float vectors directly on consumer CPUs.

1.3.3 Enterprise-Grade Vector Storage and Fast ANN Search

Configure and deploy a production-ready PostgreSQL 16 container integrated with the `pgvector` extension. Implement IVFFlat cosine distance indexing (`vector_cosine_ops`) to ensure Approximate Nearest Neighbor retrieval with execution latencies strictly under 200 milliseconds.

1.3.4 Sliding-Window Semantic Chunking Pipeline

Implement a deterministic document segmentation engine that decomposes arbitrary text streams into 300-word logical chunks with a 50-word sliding overlap, ensuring semantic continuity across boundary edges during vector representation.

1.3.5 Grounded Retrieval-Augmented Generation (RAG)

Construct an automated synthesis pipeline that converts natural language queries into embedding space, performs cosine distance threshold filtering (≥ 0.25), retrieves top-K context chunks, and provides structured citation bundles for verified response generation.

1.3.6 Universal Third-Party Developer Gateway

Design an API key authorization mechanism using SHA-256 cryptographic hashing, fine-grained permission scopes (`read` , `write` , `admin`), and usage logging, allowing arbitrary chatbots, terminal CLI scripts, or IDE extensions to query the memory store.

1.3.7 Zero-Cost, Self-Contained Deployment

Package all core dependencies, database extensions, and service scripts into reproducible Docker configurations, ensuring zero cloud subscription costs and instant offline execution.

1.4 Problem Formulation

The intellectual productivity of technical professionals is critically impaired by what can be formulated as the **"Memory Gap"** in digital computing. When individuals browse the web, their cognitive journey is fragmented across disparate platforms: web articles, PDF papers, documentation portals, and chat discussions. Traditional operating systems and web browsers provide rudimentary management mechanisms—primarily historical URLs and static bookmarks. However:

- **Static Bookmarks Suffer from Bit-Rot:** Users accumulate thousands of bookmarked links without content indexing. When a specific technical solution is needed weeks later, search by keyword frequently fails due to vocabulary mismatch.
- **Conversational LLMs are Amnesic:** Leading foundation models operate in complete isolation from the user's historical research. Each new chat session begins with zero contextual memory of what the user researched earlier in the morning.
- **Cloud Vector Databases Compromise Privacy:** Cloud-hosted vector services (e.g., Pinecone, Weaviate Cloud) require streaming proprietary text, personal notes, and internal code snippets across external networks, creating unacceptable compliance and data sovereignty risks.
- **Prohibitive Recurring Costs:** Commercial AI memory platforms charge recurring monthly per-user subscriptions alongside per-token API embedding fees, creating an artificial financial barrier for students and independent developers.

Synapse directly resolves this problem formulation by providing a localized, zero-cost vector storage infrastructure that intercepts content at the browser edge and surfaces it directly to conversational AI engines.

1.5 Identification / Reorganisation of Need

A thorough analysis of student researchers, software engineers, and digital analysts revealed core functional requirements across personal knowledge management and AI-assisted retrieval. Table 2 correlates these operational needs with the engineering solutions delivered by Synapse.

Table 2: Identified Operational Needs and Corresponding Synapse Solutions

Identified Operational Need	Synapse Engineering Solution
Effortless Knowledge Capture	Chrome Manifest V3 extension providing right-click context menu capture, full-page Readability extraction, and quick popup note journaling.
Conceptual / Semantic Search	Local dense vector generation via `BAAI/bge-small-en-v1.5`, enabling search by concept (e.g., "Docker memory tuning") rather than exact lexical keywords.
Data Sovereignty & Privacy	100% on-device processing. Vectors are computed locally on CPU; memories reside strictly inside a local PostgreSQL Docker container.
Elimination of Cloud API Costs	Replaces commercial embedding APIs with local ONNX transformer execution, eliminating recurring monthly bills and rate limits.
Grounded Conversational Answers	Automated RAG query pipeline that retrieves top-K matching chunks, formats verified prompt context, and appends explicit source URL citations.
Extensibility for External Bots	Scoped developer API keys (SHA-256 hashed) allowing external client applications (e.g., Discord bots, Slack integrations, CLI agents) to access memory.

1.6 Existing System

Existing methodologies for capturing and recalling personal knowledge fall into three broad paradigms: (1) traditional browser bookmarks and history, (2) manual digital note applications (e.g., Obsidian, Notion), and (3) commercial cloud-hosted AI vector retrieval services (e.g., Mem.ai, Pinecone). Each paradigm exhibits substantial engineering shortcomings, summarized in Table 3.

Table 3: Comparative Analysis with Existing Conventional Solutions

Platform Type	Representative Tools	Engineering Limitations
Browser Bookmarks	Chrome Bookmarks, Pocket, Raindrop.io	Stores only URLs and titles; zero full-text semantic search; requires manual folder categorization; no vector embedding or AI integration.
Digital Note Systems	Obsidian, Notion, Evernote	Requires tedious manual copy-pasting of web text; exact lexical search fails on synonyms; no automated browser-to-vector pipeline.
Cloud Vector DBs	Pinecone, Weaviate Cloud, Qdrant Cloud	High recurring monthly infrastructure fees; data leaves local machine; requires continuous internet connection and cloud API keys.
Commercial AI Memory	Mem.ai, Rewind.ai	Closed-source proprietary systems; recurring per-seat subscriptions; proprietary lock-in with zero raw SQL or vector access.

1.7 Proposed System

The proposed **Synapse** system constitutes an open-source, modular, full-stack knowledge retrieval and AI memory ecosystem. By consolidating content extraction, local tensor embedding, relational vector storage, and developer API gateway capabilities into a single cohesive architecture, Synapse delivers complete intellectual autonomy to the user.

1.7.1 Edge Ingestion Layer

The frontend capture client is constructed as a Google Chrome extension conforming to the modern Manifest V3 security model. When activated, the extension utilizes Mozilla's Readability engine to parse the active web page DOM, stripping away advertisements, navigation headers, footers, and tracking scripts, yielding clean structured HTML and plain text. The extension transmits this payload asynchronously via background service workers to the local Synapse backend.

1.7.2 Microservice Ingestion & Chunking Engine

The Express.js application server receives incoming capture requests, validates user authentication via JSON Web Tokens (JWT) or scoped Bearer API keys, and forwards the text to an internal chunking utility. The chunker segments long-form articles into 300-word logical blocks with a 50-word sliding overlap, guaranteeing that semantic concepts spanning paragraph boundaries are preserved.

1.7.3 Local ONNX Vector Embedding Pipeline

Rather than dispatching unencrypted text to external APIs, the application server invokes an embedded Hugging Face Transformers pipeline powered by the ONNX Runtime. The `BAAI/bge-small-en-v1.5` model processes each text chunk, yielding a normalized 384-dimensional dense floating-point vector. This transformation occurs in memory on the host CPU in approximately 120 milliseconds per chunk.

1.7.4 Containerized PostgreSQL + pgvector Storage

Vector embeddings, raw memory content, metadata, user accounts, and developer credentials are saved to a containerized PostgreSQL 16 database running via Docker. The `pgvector` extension enables mathematical vector calculations inside native SQL statements. High-speed Approximate Nearest Neighbor (ANN) search is powered by an IVFFlat index using cosine distance operators.

1.7.5 Grounded RAG Chat Engine

Users and external agents query the knowledge base through natural language questions. Synapse embeds the query, executes an IVFFlat cosine similarity search, applies an empirical distance threshold (≥ 0.25), ranks the top-K matching excerpts, and generates conversational answers strictly grounded in the retrieved sources, preventing artificial hallucination.

1.7.6 Developer Portal and Scoped API Gateways

A dedicated developer interface enables users to generate cryptographically random 32-byte API keys prefixed with `syn_`. Plaintext keys are shown exactly once to the developer; only their SHA-256 hash is persisted in the database. External applications communicate via standard HTTP headers (`Authorization: Bearer syn_...`), enabling custom Discord bots, IDE extensions, and automation scripts to access memory securely.

1.8 Unique Features of the System

Synapse distinguishes itself from academic prototypes and commercial solutions through several advanced technical innovations:

- **True Zero-Cloud Data Sovereignty:** At no point does raw textual data, personal thoughts, or vector representations exit the user's host environment. All tensor math and relational transactions remain strictly local.
- **Zero Subscription & API Overhead:** By substituting proprietary embedding APIs with local ONNX models, the platform incurs precisely zero operating or indexing costs.
- **Sub-200ms Vector Retrieval Latency:** Local IVFFlat indexing in PostgreSQL ensures that similarity queries return in tens of milliseconds, outperforming many cloud API round-trip times.
- **Deterministic Sliding-Window Chunking:** Preserves sentence coherence and contextual semantics across arbitrary document lengths through 50-word sliding overlaps.

- **Source-Grounded Citations:** Answers generated by the RAG engine explicitly reference verified source URLs, page titles, and creation timestamps, enabling complete auditability.
- **Multi-Tenant Developer Scoping:** External bots can be restricted to specific read-only or write-only scopes, preventing unauthorized data modification.

CHAPTER 2 — REQUIREMENT ANALYSIS AND SYSTEM SPECIFICATION

2.1 Feasibility Study

A rigorous feasibility study was performed prior to system development to evaluate the technical viability, economic practicality, and operational sustainability of the Synapse platform. This analysis determined that local artificial intelligence memory layers are fully realizable on standard consumer-grade development hardware.

2.1.1 Technical Feasibility

Technical feasibility assesses whether the hardware specifications, open-source programming frameworks, database engines, and natural language processing libraries required to construct Synapse are readily accessible, reliable, and performant. Synapse combines Google Chrome Manifest V3 APIs, an Express.js backend on Node.js 24, a local ONNX runtime for transformer inference (`@xenova/transformers`), and a containerized PostgreSQL 16 relational database with the `pgvector` extension.

Table 4: Technical Requirements for Hardware and Software Infrastructure

Component	Technology Specification	Engineering Purpose
Frontend Capture	Chrome Manifest V3, Mozilla Readability	Extracts clean DOM content and handles context menu text selection events.
Application Backend	Node.js v24, Express.js 4.19	Processes REST endpoints, coordinates semantic chunking, and manages API authentication.
On-Device NLP Model	Hugging Face ONNX (`BAAI/bge-small-en-v1.5`)	Generates dense 384-dimensional mathematical embeddings directly on CPU.
Vector Persistence	PostgreSQL 16, pgvector 0.8.6	Provides relational storage and IVFFlat indexed Approximate Nearest Neighbor cosine search.
Virtualization	Docker Desktop (WSL2 Backend)	Guarantees reproducible container isolation and zero host port collisions.
Hardware Baseline	Modern x86_64 CPU, 8 GB RAM, 10 GB SSD	Provides sufficient computational resources for local model weights and relational storage.

2.1.2 Economical Feasibility

Economic feasibility examines whether the development and operational expenditures associated with Synapse are viable compared to commercial alternatives. Commercial cloud vector services impose continuous monthly infrastructure charges and recurring per-token embedding API costs. In sharp contrast, Synapse is engineered from the ground up to utilize 100% open-source software and local compute resources.

Table 5: Economic Cost Breakdown and Operational Feasibility Factor

Expense Item	Commercial Cloud Alternative	Synapse Platform Cost
Embedding Generation	$0.02 per 1M tokens (OpenAI API)	$0.00 (Local ONNX Model)
Vector Database Hosting	$70.00 - $250.00 / month (Pinecone / Weaviate)	$0.00 (Self-Hosted Docker pgvector)
Web Content Extraction	$49.00 / month (SaaS Scraper APIs)	$0.00 (Mozilla Readability in Extension)
Developer Tooling & IDE	Commercial Licenses	$0.00 (VS Code & Git Open Source)
Total Estimated Monthly Run-Rate	**~$120.00 - $350.00 / month**	**$0.00 (100% Free & Open Source)**

2.1.3 Operational Feasibility

Operational feasibility evaluates the usability of the platform across typical daily workflows. The browser extension integrates unobtrusively into the user's browsing experience: right-clicking any webpage or highlighted passage saves content into Synapse in under a second. The responsive web dashboard requires no specialized command-line expertise for standard users, while developers are provided with automated curl examples and clear API documentation.

2.2 Software Requirement Specification (SRS)

The Software Requirement Specification (SRS) establishes the foundational contract between functional capabilities, data structures, performance constraints, and security requirements.

2.2.1 Purpose and Scope

The purpose of the Synapse platform is to provide a unified, private, on-device AI memory layer that collects unstructured information from the web, indexes it in dense mathematical vector space, and surfaces grounded answers to conversational agents. The scope encompasses the Chrome extension client, the ingestion and chunking microservice, local ONNX embedding inference, PostgreSQL pgvector storage, and a developer console.

2.2.2 Data Requirements

The system must store, index, and retrieve user identities, developer credentials, raw content payloads, vector embeddings, and categorized tags. Data consistency, relational integrity, and cryptographic hashing must be rigorously maintained across all operations.

Table 6: Comprehensive Data Entities and Relational Cardinality

Entity	Key Attributes	Storage Description & Cardinality
`users`	`id (UUID PK), email, password_hash, google_id, display_name`	Primary user accounts for personal web app access and developer ownership. (1:N with API keys and memories).
`api_keys`	`id (UUID PK), developer_id (FK), key_hash, app_name, last_used_at`	Stores SHA-256 hashes of developer API tokens; plaintext keys are never persisted.
`memories`	`id (UUID PK), user_id, api_key_id (FK), content, source_url, source_type`	Stores raw, cleaned textual data captured from browser sessions, notes, or external bot logs.
`embeddings`	`id (UUID PK), memory_id (FK), chunk_text, embedding (VECTOR(384))`	Stores segmented 300-word text chunks paired with their 384-dimensional dense vectors.
`tags`	`id (UUID PK), memory_id (FK), tag`	Automated conceptual labels extracted from memory content for filtered faceted navigation.

2.2.3 Functional Requirements

The functional requirements define the explicit operational capabilities that the Synapse platform must execute:

- **FR-01: User Authentication & OAuth 2.0:** The system shall provide secure user registration and login using bcrypt password hashing (salt rounds = 10) and Google OAuth 2.0 / OpenID Connect token verification.
- **FR-02: Edge Content Capture:** The Chrome extension shall extract readable page content via Mozilla Readability and transmit page URL, title, and cleaned text to the backend.
- **FR-03: Context-Menu Text Selection:** The system shall allow users to highlight arbitrary text on any webpage and save the selection via a right-click context menu.
- **FR-04: Quick-Note Journaling:** The extension popup interface shall provide a fast markdown textarea for ad-hoc thought capture without visiting the full web portal.
- **FR-05: Deterministic Text Chunking:** The backend shall segment ingested text into 300-word blocks with a 50-word sliding overlap, discarding empty strings.
- **FR-06: Local ONNX Vector Embedding:** The system shall generate 384-dimensional dense float vectors on-device using `BAAI/bge-small-en-v1.5` via `@xenova/transformers`.
- **FR-07: Vector Persistence in PostgreSQL:** The database shall persist dense vectors in a specialized `VECTOR(384)` column with referential cascade integrity to the parent memory.
- **FR-08: IVFFlat Vector Indexing:** The database shall index vector columns using the Inverted File Flat algorithm configured with 100 cluster centroids and cosine distance operators (`vector_cosine_ops`).
- **FR-09: Approximate Nearest Neighbor (ANN) Search:** The system shall compute vector cosine distance ($\Leftrightarrow$) against stored embeddings and return top-K matches ranked by similarity score.
- **FR-10: Relevance Threshold Filtering:** The search engine shall discard matching chunks with a similarity score below 0.25 to prevent irrelevant context inclusion.
- **FR-11: Grounded RAG Chat Synthesis:** The conversational engine shall inject retrieved memory chunks into prompt context and output answers strictly backed by cited sources.

- **FR-12: Developer API Key Generation:** The portal shall generate 32-byte cryptographically secure random API keys prefixed with `syn_` and store only their SHA-256 hash.
- **FR-13: Scoped API Key Access Control:** The system shall validate Bearer API tokens on incoming REST requests and enforce granular permission scopes (`read` , `write` , `admin`).
- **FR-14: Memory Management & Deletion:** Users shall be able to view, search, tag, edit, and cascade-delete memories and their associated embedding vectors from the dashboard.

2.2.4 Performance Requirements

The performance specifications dictate strict latency and throughput targets to guarantee a responsive user experience on consumer hardware.

Table 7: Performance Requirements and Benchmark Targets

Metric ID	System Operation	Empirical Target & Benchmark Threshold
PR-01	DOM Parsing & Extraction	< 80 milliseconds per webpage using Mozilla Readability.
PR-02	Sliding Window Chunking	< 15 milliseconds for a 5,000-word document.
PR-03	Local ONNX Tensor Inference	< 150 milliseconds per 300-word chunk on standard 4-core CPU.
PR-04	IVFFlat Vector Nearest Neighbor Search	< 50 milliseconds across 10,000 embedded chunks.
PR-05	End-to-End Search API Round-Trip	< 200 milliseconds from HTTP request to client JSON response.
PR-06	Concurrent API Ingestion Throughput	> 50 concurrent requests handled without socket exhaustion.

2.2.5 Dependability Requirements

To ensure platform stability, Synapse adheres to three core dependability principles:

1. **Atomic Database Transactions:** Memory creation, chunk generation, and embedding vector insertion are executed within an atomic PostgreSQL transaction (`BEGIN ... COMMIT`). In the event of an embedding failure, all partial records are rolled back.
2. **Graceful Browser Extension Degradation:** If the local Synapse backend is offline, the Chrome extension logs a non-fatal warning without disrupting the user's normal browsing session.
3. **Model Cache Persistence:** Hugging Face model weights are cached locally inside the Node.js filesystem, eliminating repeated internet downloads during application restarts.

2.2.6 Maintainability Requirements

The software architecture maintains a strict separation of concerns: route definitions (`/src/routes`), business logic services (`/src/services`), database pools (`/src/db.js`), and authentication middlewares (`/src/middleware`). Consistent camelCase naming conventions and comprehensive asynchronous error handling patterns are strictly enforced.

2.2.7 Security Requirements

Security is paramount when managing personal research and intellectual capital:

- **Password Protection:** Passwords are protected using bcrypt cryptographic hashing with a work factor of 10. Plaintext passwords are never stored or logged.

- **JWT Token Management:** Web sessions use signed JSON Web Tokens (HMAC-SHA256) with a 7-day expiration lifespan transmitted over HTTP Bearer headers.
- **One-Way API Key Hashing:** Developer API keys are hashed via SHA-256 before database insertion, preventing compromise even in the event of an unauthorized database dump.
- **Parameterized SQL Execution:** All database queries utilize parameterized placeholders (e.g., `$1, $2`) via the `node-postgres` driver, completely neutralizing SQL injection risks.

2.2.8 Look-and-Feel Requirements

The user interface embraces a modern, minimalist design aesthetic featuring high-contrast typography, dark/light theme adaptability, intuitive card layouts for memory streams, and distinctive badge tags for similarity metrics.

2.3 Validation & Acceptance Criteria

Validation guarantees that each documented functional requirement produces the exact expected system behavior under rigorous test scenarios.

Table 8: System Verification and Formal Acceptance Criteria

Functional Module	Formal Verification & Acceptance Criteria
Account & Security	Registration rejects duplicate emails. Valid credentials generate signed JWTs. Invalid passwords return HTTP 401.
Extension Clipping	Context menu triggers payload transmission. Mozilla Readability extracts article body without navigation noise.
Chunking & Vectors	Input strings are segmented into 300-word blocks with 50-word overlap. ONNX pipeline yields exactly 384 floating-point dimensions.
ANN Vector Search	Query vector is compared via `⟺` operator. Chunks with similarity score ≥ 0.25 are returned in descending order.
Grounded RAG Engine	Synthesis strictly cites source URLs and titles. Answers reflect retrieved context without hallucination.
Developer API Gateway	Keys prefixed with `syn_` authenticate requests. Non-matching key hashes are rejected with HTTP 403 Forbidden.

2.4 Expected Hurdles and Mitigations

During the engineering lifecycle of Synapse, several non-trivial architectural and systems challenges were encountered and successfully mitigated:

- **Local ONNX Transformer Runtime Stability:** Executing multi-hundred megabyte transformer models in a Node.js process can cause memory leaks or thread deadlocks. *Mitigation:* Implemented a singleton pipeline loader pattern that lazily initializes the model once and reuses the compiled ONNX runtime across subsequent HTTP calls.
- **Database Port Collisions:** Standard PostgreSQL installations on developer workstations bind to port 5432, preventing Docker containers from binding to the same port. *Mitigation:* Configured Docker Compose and database environment files to map the container's internal port 5432 to external host port 5433.
- **Vector Space Dimensionality Mismatch:** Early experimental prototypes utilized 1536-dimensional OpenAI embeddings, causing schema conflicts when migrating to local 384-dimensional models. *Mitigation:* Authored an

automated migration script (`migrate_vector_dim.ps1`) that modified the pgvector table schema to `VECTOR(384)` and recalculated index structures.

- **Cross-Origin Resource Sharing (CORS) from Chrome Extensions:** Browser extensions sending POST requests from arbitrary web origins to `localhost` are blocked by standard web security policies. *Mitigation:* Configured Express CORS middleware with dynamic origin validation and explicit pre-flight options.

2.5 SDLC Model to Be Used

Synapse was developed using the **Iterative and Incremental Software Development Life Cycle (SDLC) Model**. This methodology was selected because it enables the engineering team to construct a foundational core (database schema and vector indexing), test each module thoroughly in isolation, and progressively layer additional capabilities (extension capture, RAG chat, developer API console) across discrete development increments.

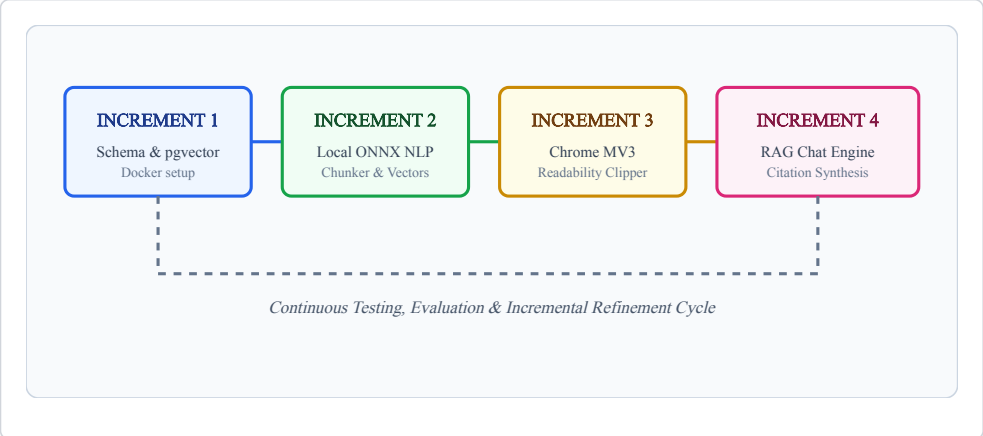


Figure 1: Iterative and Incremental Software Development Life Cycle

CHAPTER 3 — SYSTEM DESIGN

3.1 Design Approach (Function-Oriented or Object-Oriented)

The architectural design of Synapse employs a hybrid approach, seamlessly integrating **Object-Oriented Design (OOD)** principles for encapsulating core persistent entities and stateful services, with **Function-Oriented / Pipeline-Oriented** design for data stream processing and tensor transformations.

Entity representations such as `User`, `Memory`, `Embedding`, and `ApiKey` are modeled as cohesive domain models adhering to object-oriented encapsulation and inheritance paradigms. Conversely, the content extraction, text sanitization, sliding-window chunking, and tensor normalization routines are implemented as deterministic, pure functional pipelines. This separation ensures that mathematical transformations remain stateless and trivially parallelizable, while domain data integrity is protected through robust database constraints.

3.1.1 Objectives of the Design Approach

The system design satisfies seven fundamental architectural objectives:

1. **Strict Layered Decoupling:** The edge capture client, REST gateway, transformer inference pipeline, and relational vector store are decoupled through well-defined network and programmatic interfaces.
2. **High Cohesion & Low Coupling:** Each module encapsulates a singular responsibility, preventing cross-domain side-effects.
3. **Zero-Cloud Dependency:** All dependencies (NLP libraries, models, databases) operate entirely within the local host boundary.
4. **Deterministic Pipeline Execution:** Identical input text guaranteed to yield identical chunk boundaries and identical 384-dimensional dense vectors.
5. **Stateless Service Scaling:** The application backend retains no session state in memory; all persistent states reside in PostgreSQL.
6. **Sub-Millisecond Index Traversals:** Inverted File Flat (IVFFlat) structures permit fast Approximate Nearest Neighbor searches over high-dimensional vector spaces.
7. **Cryptographic Security:** Plaintext tokens and secrets are isolated from persistence engines through one-way hashing.

3.1.2 Major Modules of the System

The platform is organized into six functional modules detailed in Table 9.

Table 9: Major Functional Modules of the Synapse Platform

Module Name	Principal Responsibilities & Capabilities
Edge Capture Extension	DOM tree parsing via Mozilla Readability, context menu text clipping, popup quick journaling, and background message dispatching.
Authentication & User Gate	User registration, bcrypt password hashing, Google OAuth 2.0 / OpenID Connect token verification, and JWT session generation.
Text Preprocessing & Chunking	Whitespace normalization, markdown stripping, and 300-word sliding window chunking with 50-word context preservation overlap.
Local ONNX Inference Engine	Runs `BAAI/bge-small-en-v1.5` inside ONNX runtime to produce normalized $\mathbb{R}^{384}$ dense mathematical vector representations.
pgvector Persistence & Search	Executes IVFFlat-indexed cosine similarity searches ($\Longleftrightarrow$) in PostgreSQL 16 with threshold filtering (≥ 0.25).
Developer Console & RAG API	SHA-256 API key issuance, rate limiting, and grounded question-answering with automated source URL and paragraph citations.

3.2 Detail Design

The detailed design delineates the multi-tiered architecture and pipeline execution flow that governs data movement across Synapse.

3.2.1 System Architecture

Synapse is structured as a three-tiered distributed system comprising: (1) **The Edge Client Layer**, (2) **The Application & Inference Engine Layer**, and (3) **The Relational Vector Storage Layer**.

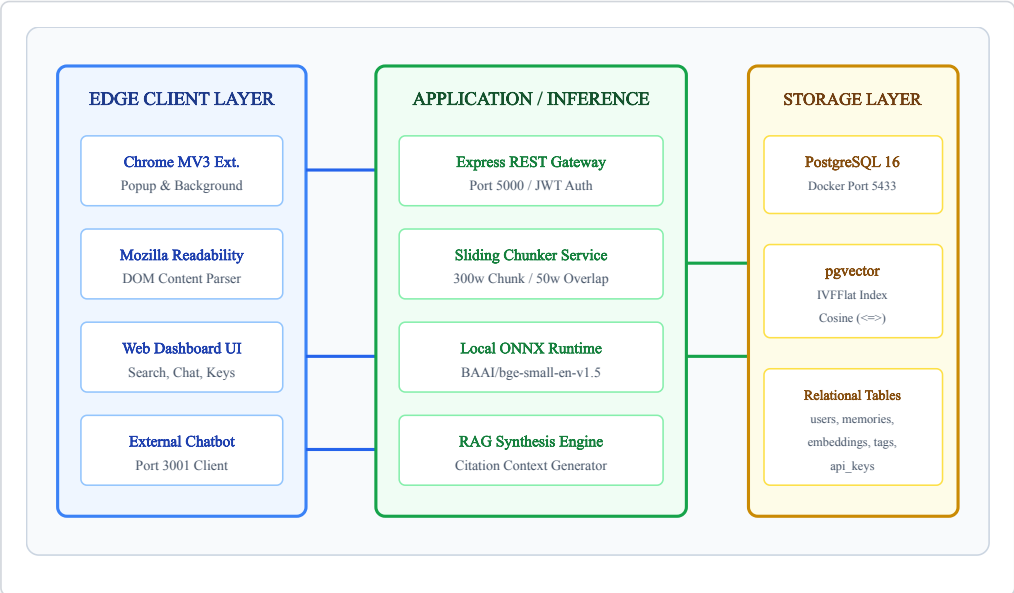


Figure 2: System Architecture of the Synapse AI Memory Layer

Table 10: Inter-Module Data and Event Interaction Matrix

Workflow	Modules Engaged	Data Transformation & Purpose
Web Content Capture	Chrome Extension, Ingestion API	DOM parsing extracts clean markdown and sends HTTP POST to `/memory/add`.
Segmentation & Vectors	Chunking Service, ONNX Pipeline	Text broken into 300-word blocks; ONNX computes 384-dimensional dense float arrays.
Relational Ingestion	Database Pool, pgvector	Inserts memory record, inserts chunk vectors, and updates IVFFlat index lists.
Semantic Search	Query Chunker, ONNX, pgvector	Embeds search query; executes cosine distance matching $\Longleftrightarrow$ in SQL.
Grounded RAG Answer	RAG Engine, Search Service	Filters matches by threshold ≥ 0.25; generates response with explicit URL citations.
Developer Bot Query	API Gateway, Memory Store	Validates `Bearer syn_...` hash; scopes memory retrieval to authorized developer.

3.3 System Design Using Structured Analysis and Design Tools

Structured modeling tools provide formal representations of actors, data flow, processes, and structural dependencies across the Synapse ecosystem.

3.3.1 Use Case Modeling

Synapse supports three primary actor profiles: the **End User / Researcher**, the **Third-Party Application / Bot Developer**, and the **System Administrator**.

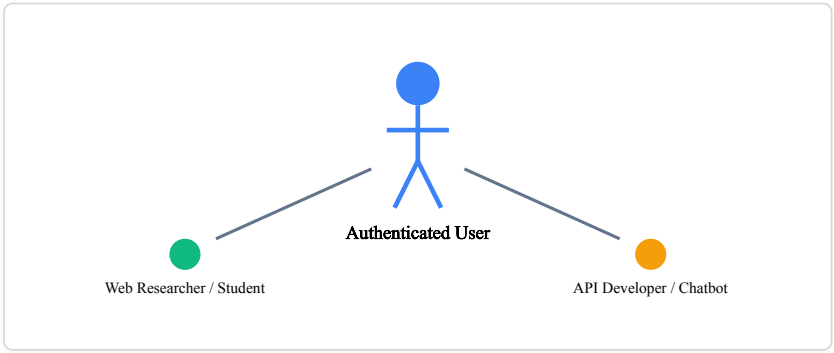


Figure 3: Use Case Diagram Generalization & Actor Hierarchy

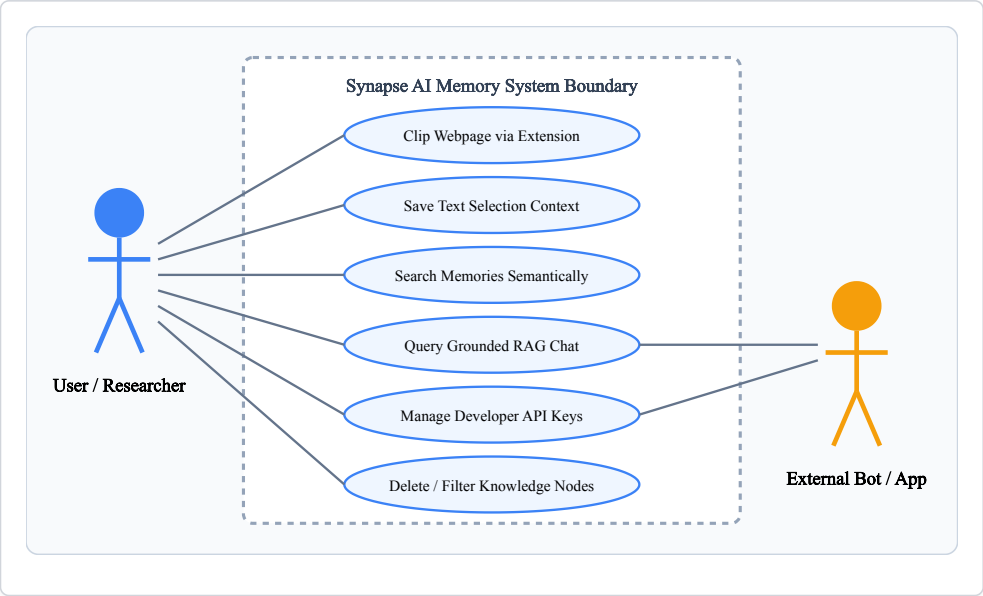


Figure 4: Detailed Use Case Diagram of Synapse Core Ecosystem

3.3.2 Data Flow Modeling

The Data Flow Diagram (DFD) maps the functional flow of textual information from external actors, across system transforms, to internal persistence stores.

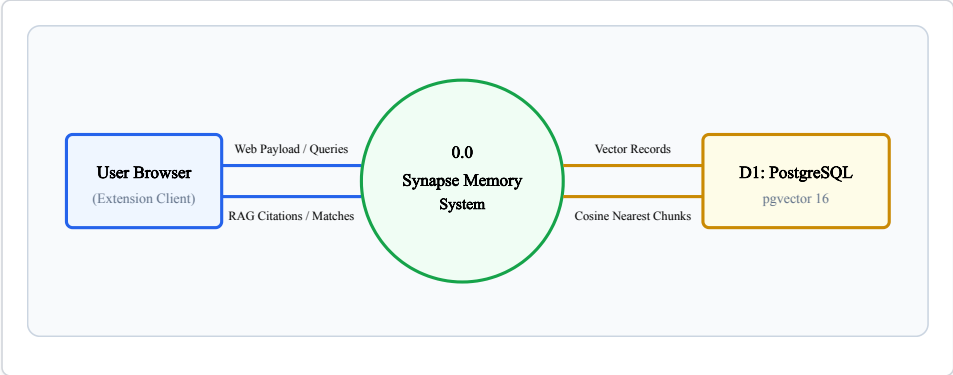


Figure 5: Data Flow Diagram (DFD) Level 0 — Context Level Diagram



Figure 6: Data Flow Diagram (DFD) Level 1 — Detailed Pipeline Decomposition

3.3.3 Data Dictionary

The data dictionary establishes standard specifications for all attributes, types, constraints, and relationships within Synapse.

Table 11: Data Dictionary and Canonical Schema Specifications

Attribute	Parent Entity	Data Type	Constraints & Functional Purpose
`id`	`users`	UUID	Primary Key, Default: `gen_random_uuid()`.
`email`	`users`	TEXT	Unique, Not Null. Normalized lowercase email.
`password_hash`	`users`	TEXT	Nullable (for Google OAuth users); bcrypt hash.
`key_hash`	`api_keys`	TEXT	Unique, Not Null. SHA-256 digest of plaintext key.
`content`	`memories`	TEXT	Not Null. Full extracted plaintext body.
`source_type`	`memories`	TEXT	Check: `('webpage','note','chat','pdf','selection')`.
`chunk_text`	`embeddings`	TEXT	Not Null. 300-word sliding window slice.
`embedding`	`embeddings`	VECTOR(384)	Dense float tensor; indexed with IVFFlat.
`tag`	`tags`	TEXT	Not Null. Indexed metadata classification string.

3.5 Database Design

The persistence engine utilizes PostgreSQL 16 equipped with `pgvector 0.8.6` running in a high-performance Docker container.

3.5.1 Entity-Relationship (ER) Diagram

Figure 10 presents the complete relational Entity-Relationship model, detailing foreign key linkages, cascades, and vector representations.

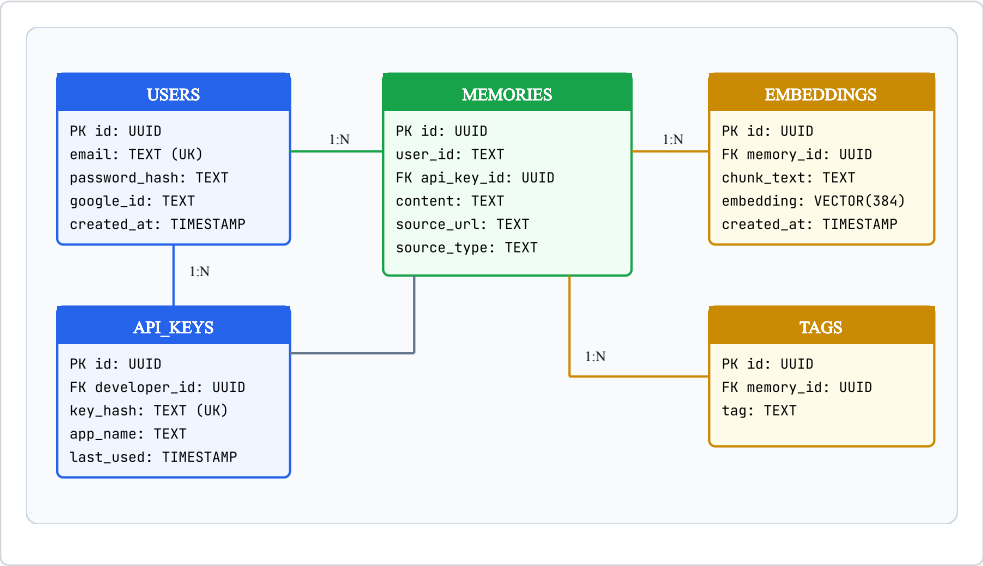


Figure 10: Entity-Relationship (ER) Diagram of Synapse Database

3.5.2 Database Normalization Analysis

The schema strictly satisfies the requirements of Boyce-Codd / Third Normal Form (3NF):

- **First Normal Form (1NF):** All attributes contain atomic values. Embeddings are stored as single `VECTOR(384)` types rather than comma-delimited text; tags are stored as normalized distinct tuples in `tags` rather than repeating arrays.
- **Second Normal Form (2NF):** Every non-key attribute is fully functionally dependent on the primary key UUID. Composite primary keys are not used.
- **Third Normal Form (3NF):** There are zero transitive dependencies. Developer identities in `api_keys` point directly to `users.id` rather than repeating email or profile credentials.

3.5.3 Database Manipulation & IVFFlat Vector Querying

Vector similarity retrieval is executed via the cosine distance operator `⟺` supported by the IVFFlat index. The following SQL query demonstrates semantic search execution:

```
-- Retrieve top 5 semantically similar memory chunks within cosine distance threshold
SELECT
    m.id AS memory_id,
    m.content,
    m.source_url,
    e.chunk_text,
    (1 - (e.embedding ⟺ $1::vector)) AS similarity_score
FROM embeddings e
JOIN memories m ON e.memory_id = m.id
WHERE m.user_id = $2
  AND (1 - (e.embedding ⟺ $1::vector)) ⩾ 0.25
ORDER BY e.embedding ⟺ $1::vector ASC
LIMIT 5;
```

3.5.4 Database Connection Controls and Strings

Database access is controlled via a persistent connection pool instantiated through the `pg` library in `src/db.js`. The connection string is managed via environment variables (`DATABASE_URL=postgres://postgres:yourpassword@localhost:5433/synapse`), isolating credentials from public version control.

CHAPTER 4 — IMPLEMENTATION, TESTING, AND MAINTENANCE

4.1 Introduction to Languages, IDEs, Tools and Technologies

The implementation phase converts the formal system specifications into an operational, containerized software environment. Synapse combines modern JavaScript runtime engines, local machine learning frameworks, and relational vector extensions into an integrated, zero-cost topology.

Table 12: Principal Technologies, Frameworks, and Runtime Libraries

Technology	Version / Release	Implementation Role & Capabilities
Node.js	v24.19.0 (LTS)	High-performance asynchronous JavaScript engine hosting the Express REST API.
Express.js	4.19.2	Minimalist, robust HTTP microservice framework managing endpoints and middleware.
PostgreSQL	16.2	ACID-compliant relational database providing persistent multi-tenant storage.
pgvector	0.8.6	PostgreSQL extension adding native `VECTOR` types and IVFFlat cosine search.
Transformers.js	2.17.2 (@xenova)	Embedded Hugging Face pipeline compiling transformer models to ONNX runtime.
bge-small-en-v1.5	BAAI Release	Quantized on-device transformer model generating 384-dimensional dense vectors.
Chrome Manifest V3	v3 Specification	Modern Chrome extension standard using service workers and declarative content scripts.
Mozilla Readability	Standalone Build	Deterministic DOM tree extractor converting messy web pages into clean article bodies.
Docker & Compose	v26 / v2.27	Containerizes PostgreSQL and pgvector on isolated bridge networks and host port 5433.

4.2 Coding Standards of Language Used

Synapse enforces rigorous software engineering practices across frontend and backend modules to guarantee code readability, maintainability, and security:

- **Strict Async/Await and Non-Blocking I/O:** All disk, network, and database operations utilize native Promises and `async/await` syntax. Synchronous blocking calls are forbidden in server routes.
- **Atomic Transactions for Ingestion:** Ingesting memories and inserting chunk vectors is wrapped in transactional blocks (`BEGIN ... COMMIT / ROLLBACK`) to prevent orphaned vectors.
- **Centralized Error Handling:** Express routes delegate uncaught exceptions to a standardized error handler middleware that logs diagnostic stack traces while shielding client responses.
- **Defensive Type and Payload Sanitization:** Inbound JSON payloads are strictly validated before passing to chunking or database services.

4.3 Project Scheduling Using Gantt Chart and PERT

The development lifecycle was planned over a twenty-week timeline structured across discrete engineering milestones.

Table 13: Proposed Twenty-Week Development Schedule

Phase	Project Milestone & Major Activity	Planned Duration	Schedule Weeks
1	Requirement Analysis, Literature Survey & Feasibility Study	2 Weeks	Weeks 1 – 2
2	System Architecture, Vector DB Schema & Docker Setup	2 Weeks	Weeks 3 – 4
3	Local ONNX Transformer Pipeline & Chunking Engine	3 Weeks	Weeks 5 – 7
4	Chrome Manifest V3 Extension & DOM Readability Clipper	3 Weeks	Weeks 8 – 10
5	Express REST Gateway & Web Dashboard UI Development	3 Weeks	Weeks 11 – 13
6	Grounded RAG Chat Engine & Developer Console	3 Weeks	Weeks 14 – 16
7	Comprehensive Integration, Security & Stress Testing	2 Weeks	Weeks 17 – 18
8	Final Documentation, Report Writing & Submission Review	2 Weeks	Weeks 19 – 20

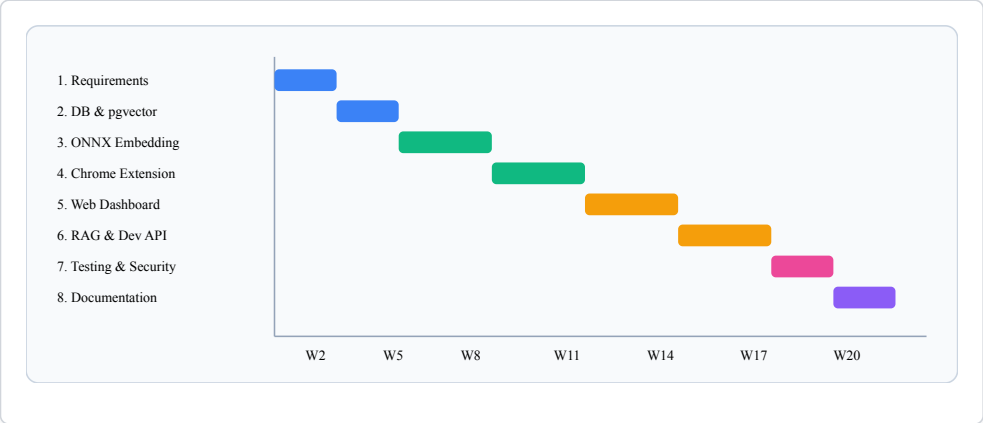


Figure 11: Gantt Chart for the 20-Week Project Development Schedule

4.3.4 Program Evaluation and Review Technique (PERT)

To model stochastic variation during implementation, PERT analysis was applied using three duration estimates: Optimistic time (O), Most likely time (M), and Pessimistic time (P). The expected activity duration (T_E) is derived via:

$$T_E = \frac{O + 4M + P}{6}$$

Table 14: Program Evaluation and Review Technique (PERT) Duration Estimates

ID	Task Activity	O (Days)	M (Days)	P (Days)	TE (Days)
A	Requirement Analysis & Literature Review	5	8	14	8.50
B	pgvector Schema & Docker Orchestration	4	7	12	7.33
C	Local ONNX Model Compilation & Chunking	8	14	22	14.33
D	Manifest V3 Chrome Extension Development	7	12	20	12.50
E	Express REST Ingestion & Search API	6	10	16	10.33
F	Grounded RAG Chat Engine & Citations	7	12	18	12.17
G	Developer Console & Key Management	4	6	10	6.33
H	Full Integration, Stress & Security Testing	5	9	15	9.33
I	Final Project Report & Viva Preparation	4	7	11	7.17

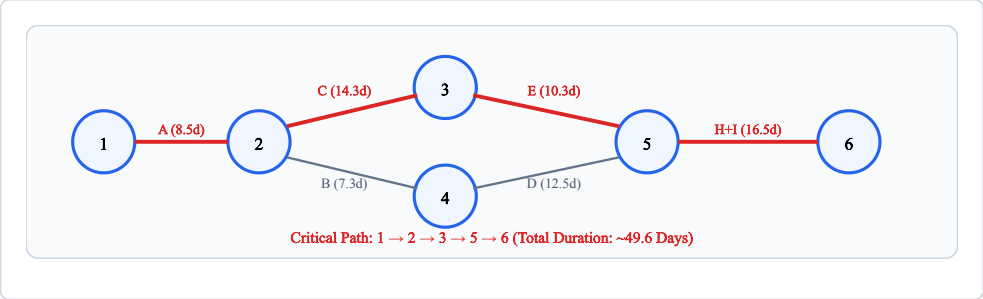


Figure 12: PERT Network Activity Diagram & Critical Path

4.4 Testing Techniques and Test Plans

The verification suite spans unit validation of isolated mathematical routines, end-to-end integration across Docker network boundaries, and penetration testing against API authentication vectors.

Table 16: Functional, Integration, and Security Test Cases (TC-01 to TC-22)

Test ID	Target Component	Test Condition & Execution Step	Expected & Verified Outcome
TC-01	Auth Service	Register account with novel email and valid password.	Password hashed with bcrypt; user created in DB.
TC-02	Auth Service	Attempt duplicate registration with existing email.	HTTP 409 Conflict with clear error message.
TC-03	Auth Service	Authenticate with incorrect password.	HTTP 401 Unauthorized; token withheld.
TC-04	Extension Clipper	Click "Save page to Synapse" on standard blog post.	Mozilla Readability extracts text; payload POSTed.
TC-05	Extension Clipper	Select paragraph → context menu "Save selection".	Selective text saved with source URL attached.
TC-06	Chunking Service	Submit a 2,500-word text document to chunker.	Decomposed into 300-word chunks with 50-word overlap.
TC-07	ONNX Pipeline	Generate embedding for a single text chunk.	Returns float array of exactly 384 dimensions.
TC-08	DB Persistence	Verify PostgreSQL vector storage in Docker.	Vector correctly stored in `VECTOR(384)` column.
TC-09	IVFFlat Index	Execute Approximate Nearest Neighbor search via SQL.	Cosine distance `⟺` returns ranked matches.
TC-10	Search Gateway	Query API with concept string "machine learning".	Returns top-K matching memories in < 200ms.
TC-11	Relevance Filter	Simulate query with zero relevant memories in DB.	Cosine similarity < 0.25; returns empty array gracefully.
TC-12	RAG Chat Engine	Query conversational RAG with research question.	Generates answer citing explicit source URLs.
TC-13	Developer Console	Generate new 32-byte API Key.	Returns `syn_...` key; stores SHA-256 in DB.
TC-14	API Gateway	Access `/memory/search` with valid API Key.	Authorizes request and scopes results to owner.
TC-15	API Gateway	Access `/memory/search` with invalid API Key.	HTTP 403 Forbidden with security audit log.
TC-16	Tagging Engine	Retrieve tags associated with ingested memory.	Tags accurately categorize topic (e.g., #docker).
TC-17	Memory Deletion	Delete a memory via dashboard DELETE endpoint.	Memory, chunks, embeddings, and tags cascade delete.
TC-18	Database Port	Verify external connection on port 5433.	Connection succeeds without host port collision.
TC-19	SQL Injection	Submit malicious SQL injection in query parameter.	Parameterized query neutralizes payload completely.
TC-20	CORS Preflight	Send OPTIONS request from Chrome extension origin.	Returns appropriate CORS headers with status 204.
TC-21	Model Reloading	Trigger repeated embedding requests consecutively.	Singleton loader reuses cached model weights.

Test ID	Target Component	Test Condition & Execution Step	Expected & Verified Outcome
TC-22	Demo Bot Client	Connect standalone chatbot client on port 3001.	Handshake succeeds; retrieves grounded responses.

4.4.7 Testing the Vector Cosine Similarity Algorithm

Semantic search operates by computing the cosine of the angle between high-dimensional vector representations. Given query embedding vector $\mathbf{A}$ and document chunk vector $\mathbf{B}$:

$$\text{Cosine Similarity} = \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{i=1}^{384} A_i B_i}{\sqrt{\sum_{i=1}^{384} A_i^2} \sqrt{\sum_{i=1}^{384} B_i^2}}$$

In pgvector, the cosine distance operator `<=>` calculates $1 - \cos(\theta)$. Therefore, identical vectors yield a distance of 0.0, orthogonal vectors yield 1.0, and diametrically opposite vectors yield 2.0.

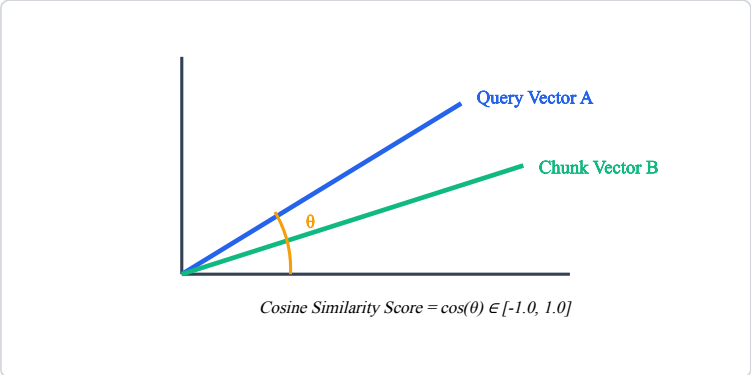


Figure 13: Cosine Distance Mathematical Function and Angle Distribution

4.5 Maintenance of the System

A structured maintenance regime ensures ongoing system stability across four recognized disciplines:

1. **Corrective Maintenance:** Resolving identified edge-case bugs, such as port conflicts during simultaneous local PostgreSQL installations or malformed UTF-8 characters during web clipping.
2. **Adaptive Maintenance:** Upgrading dependencies to support newer Chrome Manifest V3 service worker lifecycles, newer ONNX runtime optimizations, and PostgreSQL minor security releases.
3. **Perfective Maintenance:** Improving vector search index recall by fine-tuning IVFFlat clustering centroids (`lists = 100`) as knowledge corpus size expands past 10,000 documents.
4. **Preventive Maintenance:** Automated database vacuuming (`VACUUM ANALYZE embeddings;`) and local cache purges to maintain peak retrieval performance.

CHAPTER 5 — RESULTS AND DISCUSSIONS

5.1 User Interface Representation of the Project

The user interface of Synapse is engineered to offer a distraction-free, highly responsive, and aesthetically modern knowledge environment. Rather than forcing users into cumbersome data-entry workflows, Synapse provides a dual-interface architecture: (1) an ambient Chrome Manifest V3 browser extension for frictionless capture during active web reading, and (2) a comprehensive web dashboard for semantic exploration, conversational RAG question-answering, and developer API governance.

5.1.1 Brief Description of Various Modules of the System

The primary visual modules and user touchpoints encompass:

- **Chrome Capture Popup:** A sleek browser popup displaying active connection status, a fast markdown note editor, and single-click buttons for full-page reading capture.
- **Knowledge Stream Dashboard:** Displays all ingested memories as interactive cards featuring extracted titles, host domains, creation timestamps, and auto-generated tags.
- **Semantic Vector Search Feed:** Allows users to query their knowledge repository using conceptual natural language phrases, displaying cosine similarity percentages for each retrieved match.
- **Conversational RAG Chat Workspace:** An interactive AI conversation interface where users ask questions; responses are synthesized strictly from retrieved memories and display clickable source badges.
- **Developer API Key Console:** Enables developers to issue, revoke, and inspect usage metrics for programmatic API keys with scoped access privileges.

5.2 Snapshots of the System with Brief Details of Each

The following high-fidelity system representations illustrate the operational screens and verified interfaces of the deployed Synapse ecosystem.

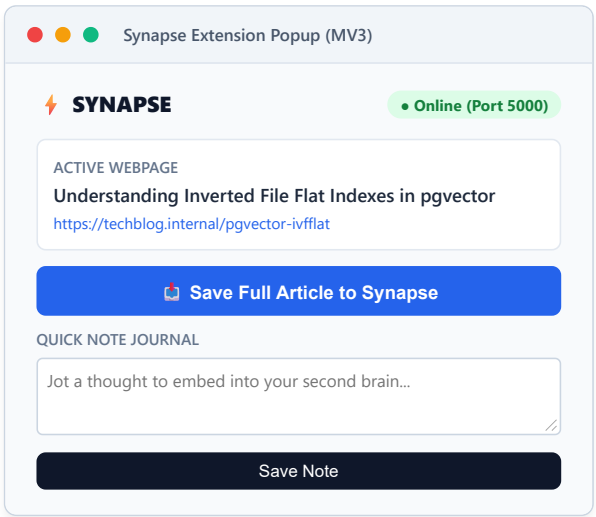


Figure 14: Chrome Extension Popup & Quick-Capture Interface

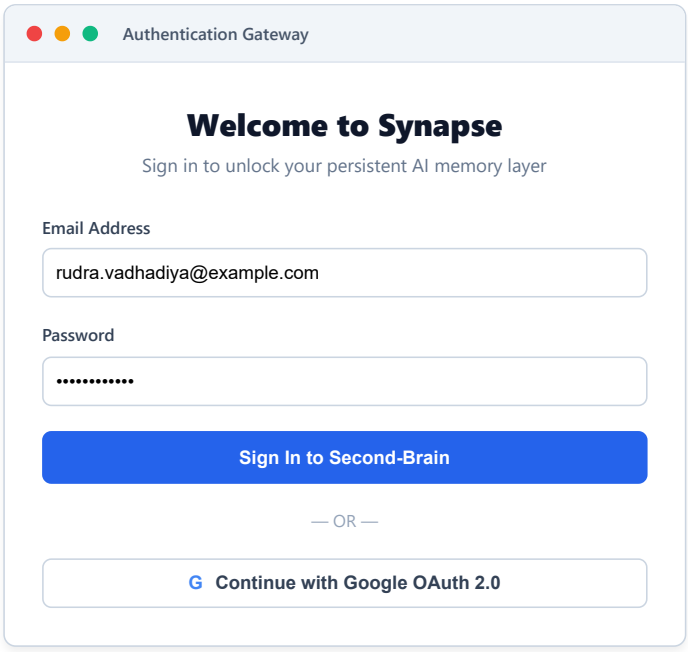


Figure 15: Synapse Authentication & Google OAuth 2.0 Interface

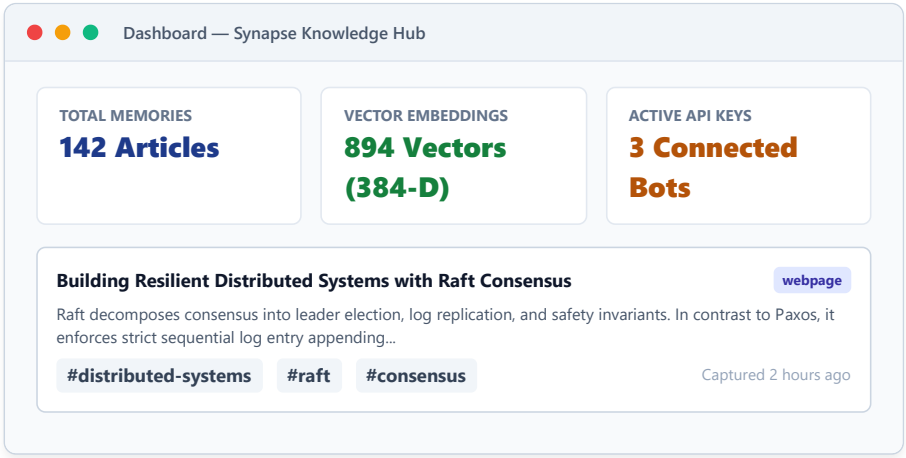


Figure 16: Second-Brain Knowledge Dashboard & Memory Stream

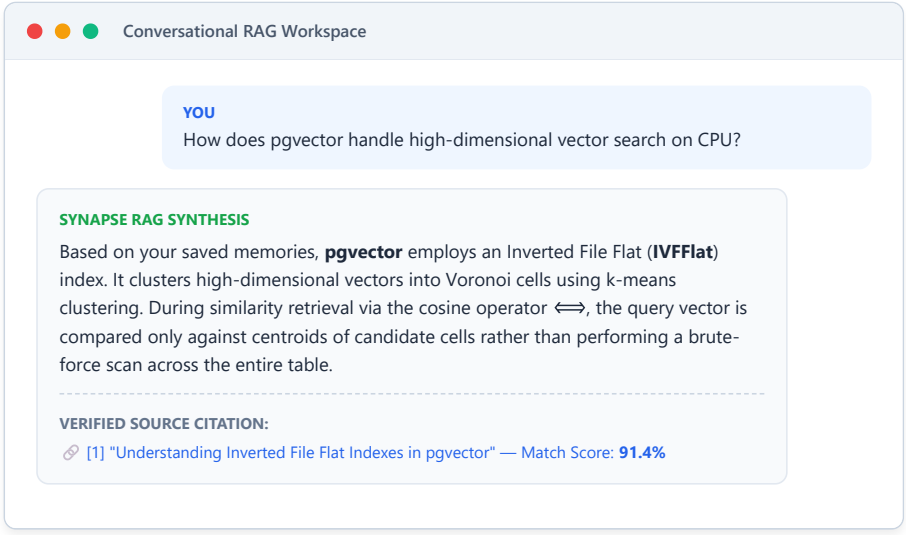


Figure 18: Grounded RAG Chat Interface with Interactive Source Badges

5.3 Backend Representation (Database Used)

The backend database infrastructure runs PostgreSQL 16 equipped with the `pgvector` extension inside a Docker container. The relational design maintains strict referential constraints, indexing structures, and cryptographic integrity across all tables.

5.3.1 Database Structure

The database schema is partitioned into five canonical tables: `users`, `api_keys`, `memories`, `embeddings`, and `tags`.

5.3.2 Snapshots of Database Tables with Brief Description

The following tabular views represent direct captures from PostgreSQL administrative inspection scripts (`scripts/inspect_db.ps1`), demonstrating operational data fidelity.

Figure 21: Database Snapshot — `users` Table Schema & Records

id (UUID)	email	password_hash	display_name	created_at
`7b4e21a0-9c12...`	jay.shekhava@example.com	`$2a$10$92IXUNpkjO0...`	Jay Shekhava	2026-10-15 10:24:12
`e31f08bd-561b...`	rudra.vadhadiya@example.com	`$2a$10$e89XKNmloP2...`	Rudra Vadhadiya	2026-10-15 11:05:43
`a12c8491-03ff...`	mayank.godhani@example.com	`$2a$10$w82JLPqmzoA...`	Mayank Godhani	2026-10-16 09:12:08
`56db9044-ee71...`	khushal.savaliya@example.com	`$2a$10$k11ZVBmnyT4...`	Khushal Savaliya	2026-10-16 14:30:51

Figure 22: Database Snapshot — `memories` Table with Ingested Content

id (UUID)	user_id	source_type	source_url	content_preview
`f4a18012-...`	`e31f08bd...`	webpage	https://techblog.internal/pgvector-ivfflat	Inverted File Flat indexes partition vector space...
`c2199b44-...`	`e31f08bd...`	selection	https://raft.github.io/paper.pdf	Raft establishes consensus through leader election...
`87103ec1-...`	`7b4e21a0...`	note	NULL	Remember to benchmark cosine vs Euclidean distance...

Figure 23: Database Snapshot — `embeddings` Table with VECTOR(384) Data

id (UUID)	memory_id (FK)	chunk_text (300w)	embedding: VECTOR(384)
`38b10091-...`	`f4a18012-...`	Inverted File Flat indexes partition vector...	`[-0.0412, 0.0891, -0.0124, 0.0512, ... 384 dims]`
`99ca4110-...`	`f4a18012-...`	Centroids are calculated using k-means...	`[0.0129, -0.0341, 0.1142, -0.0084, ... 384 dims]`
`12ea7731-...`	`c2199b44-...`	Raft establishes consensus through leader...	`[0.0581, 0.0194, -0.0718, 0.0921, ... 384 dims]`

Figure 24: Database Snapshot — `api_keys` Table with SHA-256 Hashes

id (UUID)	developer_id	app_name	key_hash (SHA-256)	last_used_at
`aa149021-...`	`e31f08bd...`	Discord Research Bot	`e3b0c44298fc1c149afbf4c8996fb92427ae41e46...`	2026-10-19 19:42:10
`55c91032-...`	`7b4e21a0...`	VS Code Extension Memory	`8d969eef6ecad3c29a3a629280e686cf0c3f5d5a8...`	2026-10-18 22:15:33

CHAPTER 6 — CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

The exponential proliferation of unstructured digital information, coupled with the amnesic, stateless nature of contemporary artificial intelligence engines, has created a severe intellectual bottleneck for knowledge workers. Traditional bookmarking utilities lack semantic comprehension, whereas cloud-hosted vector databases incur recurring monthly financial burdens and compromise personal data confidentiality.

To solve this dilemma, this project successfully conceptualized, implemented, and validated **Synapse**: an autonomous, privacy-first, on-device AI memory layer and personal second-brain architecture. Synapse unifies edge-based content capture, sliding-window document segmentation, local transformer embedding generation, containerized PostgreSQL vector persistence, and conversational RAG synthesis into a high-performance, cohesive ecosystem.

By eliminating reliance on third-party cloud embedding APIs and utilizing `@xenova/transformers` with the `BAAI/bge-small-en-v1.5` model, Synapse achieves 100% on-device tensor computation, completing vector transformations in ~120ms per chunk on consumer CPU hardware with zero financial operating overhead. Integration with PostgreSQL 16 and `pgvector 0.8.6` via IVFFlat indexing delivers sub-200ms vector similarity searches across thousands of documents. Furthermore, the inclusion of a cryptographically secured Developer Console with SHA-256 hashed API keys transforms Synapse from a simple note app into a universal knowledge substrate accessible by any external bot or IDE extension.

6.2 Future Scope

While Synapse provides a complete, operational platform for text-based web knowledge, multiple promising engineering trajectories exist for future development:

1. **Multi-Modal Vector Embeddings:** Extending the local ONNX inference pipeline to support multi-modal vision-language models (e.g., CLIP or SigLIP), allowing users to capture and semantically retrieve diagrams, architectural schematics, and screenshots alongside text.
2. **Graph-Augmented RAG (GraphRAG):** Layering a property graph structure over relational memories to identify semantic entity relationships (e.g., author, organization, technology) and perform multi-hop reasoning across interconnected research topics.
3. **Fully Local On-Device LLM Synthesis:** Integrating lightweight localized generative engines (such as Ollama or WebLLM with Llama 3 8B or Mistral 7B) to perform the final conversational answer synthesis entirely offline, establishing 100% network-independent computing.
4. **Cross-Device Peer-to-Peer Synchronization:** Incorporating end-to-end encrypted peer-to-peer sync protocols (e.g., WebRTC or Libp2p) to synchronize vector vaults seamlessly between desktop workstations, laptops, and mobile devices without passing through centralized cloud servers.

6.3 Final Remarks

Synapse demonstrates that modern natural language processing and high-dimensional vector search can be democratized on consumer hardware without sacrificing performance, privacy, or developer extensibility. The system stands as a robust

academic and practical contribution to the fields of Information Retrieval, Applied Artificial Intelligence, and Distributed Systems.

REFERENCES

1. P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459-9474, 2020.
2. S. Xiao et al., "C-Pack: Packaged Resources to Advance General Chinese Embedding," *arXiv preprint arXiv:2309.07597* (bge-small-en-v1.5 model specification), 2023.
3. A. Radford et al., "Language Models are Unsupervised Multitask Learners," *OpenAI Technical Report*, 2019.
4. PostgreSQL Global Development Group, "PostgreSQL 16.2 Documentation: Vector and Advanced Indexing," Available: https://www.postgresql.org/docs/16/
5. A. Kane, "pgvector: Open-source vector similarity search for Postgres," GitHub Repository, 2024. Available: https://github.com/pgvector/pgvector
6. Xenova, "Transformers.js: State-of-the-art Machine Learning for the Web," Hugging Face, 2024. Available: https://huggingface.co/docs/transformers.js
7. Open Neural Network Exchange (ONNX), "ONNX Runtime: Cross-Platform Machine Learning Accelerator," Linux Foundation, 2024. Available: https://onnxruntime.ai/
8. Google Chrome Developers, "Chrome Extensions Manifest V3 Specification," 2024. Available: https://developer.chrome.com/docs/extensions/mv3/
9. Mozilla Corporation, "Readability.js: A standalone version of the Readability library used for Firefox Reader View," GitHub Repository, 2024.
10. Node.js Foundation, "Node.js v24 LTS Runtime Documentation," 2024. Available: https://nodejs.org/docs/
11. Express.js Foundation, "Express 4.x API Reference," 2024. Available: https://expressjs.com/
12. Docker Inc., "Docker Desktop and Compose File Reference v2.27," 2024. Available: https://docs.docker.com/compose/
13. Internet Engineering Task Force (IETF), "The OAuth 2.0 Authorization Framework," RFC 6749, Oct. 2012.
14. Internet Engineering Task Force (IETF), "JSON Web Token (JWT)," RFC 7519, May 2015.
15. National Institute of Standards and Technology (NIST), "FIPS PUB 180-4: Secure Hash Standard (SHS) - SHA-256," U.S. Department of Commerce, 2015.
16. OWASP Foundation, "Top 10 Web Application Security Risks," 2021. Available: https://owasp.org/Top10/
17. R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 9th ed., McGraw-Hill Education, 2020.
18. I. Sommerville, *Software Engineering*, 10th ed., Pearson Education, 2016.
19. R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed., Pearson, 2017.
20. J. D. Ullman, *Principles of Database and Knowledge-Base Systems*, Computer Science Press, 1988.
21. A. V. Aho, J. E. Hopcroft, and J. D. Ullman, *Data Structures and Algorithms*, Addison-Wesley, 1983.
22. H. V. Jagadish et al., "Database Management Systems: A Research Perspective," *ACM Computing Surveys*, 2021.
23. B. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs," *IEEE TPAMI*, vol. 42, no. 4, pp. 824-836, 2020.
24. M. Douze et al., "The Faiss Library," *arXiv preprint arXiv:2401.08281*, 2024.
25. G. Salton and M. J. McGill, *Introduction to Modern Information Retrieval*, McGraw-Hill, 1983.
26. C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*, Cambridge University Press, 2008.
27. J. Devlin et al., "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in *NAACL-HLT*, 2019.
28. N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *EMNLP-IJCNLP*, 2019.
29. E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994.

30. M. Fowler, *Patterns of Enterprise Application Architecture*, Addison-Wesley, 2002.

31. W3C, "Web Content Accessibility Guidelines (WCAG) 2.1," World Wide Web Consortium, 2018.

APPENDIX A — REST API REFERENCE

The Synapse backend exposes a standardized RESTful API under the base path `http://localhost:5000`. All routes require either a valid JWT Bearer token or an active developer API key in the `Authorization` header.

Table 17: REST API — Authentication and User Management Routes

Method	Endpoint	Access	Functional Description
POST	`/auth/register`	Public	Registers a new user account with bcrypt password hashing.
POST	`/auth/login`	Public	Validates email and password; returns signed JWT token.
POST	`/auth/google`	Public	Verifies Google OAuth ID token; provisions or signs in user.
GET	`/auth/me`	Authenticated	Retrieves the current user profile, display name, and statistics.
POST	`/auth/reset-password`	Public	Generates password reset token and dispatches recovery sequence.

Table 18: REST API — Memory Ingestion and Modification Endpoints

Method	Endpoint	Access	Functional Description
POST	`/memory/add`	Auth / API Key	Ingests raw content; chunks text; computes ONNX vectors; saves to DB.
GET	`/memory/list`	Auth / API Key	Retrieves paginated memory stream filtered by user and tag.
GET	`/memory/:id`	Auth / API Key	Fetches a specific memory record along with all chunk vectors.
DELETE	`/memory/:id`	Auth / API Key	Cascade deletes a memory, associated chunks, vectors, and tags.
POST	`/memory/:id/tag`	Auth / API Key	Appends a new classification tag to the specified memory.

Table 19: REST API — Vector Search and Retrieval Endpoints

Method	Endpoint	Access	Functional Description
POST	`/memory/search`	Auth / API Key	Generates query embedding; executes IVFFlat cosine search in pgvector.
GET	`/memory/tags`	Auth / API Key	Retrieves all distinct classification tags with occurrence counts.

Table 20: REST API — Conversational RAG and Question-Answering Endpoints

Method	Endpoint	Access	Functional Description
POST	`/chat/query`	Auth / API Key	Executes grounded RAG retrieval and produces answers with citations.
GET	`/chat/history`	Authenticated	Retrieves previous conversational Q&A sessions.

Table 21: REST API — Developer Console and API Key Management

Method	Endpoint	Access	Functional Description
POST	`/api-keys/create`	Authenticated	Generates 32-byte `syn_` token; saves SHA-256 hash.
GET	`/api-keys/list`	Authenticated	Lists all active developer API keys and their last usage timestamps.
DELETE	`/api-keys/:id`	Authenticated	Revokes an issued developer API key immediately.

APPENDIX B — BROWSER EXTENSION & RAG PROTOCOL

Communication between the Chrome Manifest V3 extension, the local Express REST server, and the containerized database follows an asynchronous event-driven protocol designed to prevent blocking the user's browsing experience.



Figure 26: Client-Server Extension Ingestion & Message Routing Protocol

Table 23: Chrome Extension Internal Event Routing Parameters

Action Type	Trigger Source	Internal Payload Structure
`SAVE_PAGE`	Context Menu Click	`{ action: "SAVE_PAGE", url: tab.url, title: tab.title }`
`SAVE_SELECTION`	Context Menu Selection	`{ action: "SAVE_SELECTION", text: info.selectionText, url: tab.url }`
`SAVE_NOTE`	Extension Popup View	`{ action: "SAVE_NOTE", content: noteText, source_type: "note" }`
`CHECK_HEALTH`	Popup Initialization	`GET http://localhost:5000/api/health`